



# Shortest characteristic factors of a deterministic finite automaton and computing its positive position run by pattern set matching

Jan Janoušek<sup>1</sup> · Štěpán Plachý<sup>2</sup>

Received: 2 May 2024 / Accepted: 3 March 2025 / Published online: 10 September 2025

© The Author(s) 2025

## Abstract

Given a deterministic finite automaton (DFA)  $A$ , we present a simple algorithm for constructing deterministic finite automata that accept the shortest forbidden factors, the shortest forbidden prefixes, the shortest forbidden suffixes, the shortest forbidden words, the shortest allowed suffixes, and the shortest allowed words of the automaton  $A$ . We refer to these sets as the shortest characteristic factors of the automaton  $A$ . If the given automaton is local, and therefore the language it accepts is strictly locally testable, the sets of its shortest characteristic factors are finite, and these automata are acyclic. Otherwise, they accept infinite languages. This approach simplifies existing methods for the extraction of forbidden factors, allows the extraction of more types of characteristic factors, and also generalizes the extraction for all classes of DFAs. Furthermore, we demonstrate that this type of extraction can be used for a sublinear run of an automaton for certain inputs. We define a positive position run of a deterministic finite automaton, representing all positions in an input string where the automaton reaches a final state. Finally, we present an algorithm for computing the positive position run of the automaton, which utilizes pattern set matching of its shortest forbidden factors and its shortest forbidden or allowed suffixes, provided that the sets are finite. We showcase the computation of the positive position run of a local automaton using backward pattern set matching, which can achieve sublinear time.

## 1 Introduction

Finite automaton (FA) is a fundamental model of computation, widely studied in Computer Science and used in many applications [1, 2].

The FA can also be used as a useful formalism in the process of creating a new algorithm for a regular language problem [3–5], especially because of its simplicity, efficiency, and the availability of many operations on FA, such as determinization, minimization or construction

---

✉ Štěpán Plachý  
stepan.plachy@cs.umu.se

Jan Janoušek  
Jan.Janousek@fit.cvut.cz

<sup>1</sup> Department of Theoretical Computer Science, Faculty of Information Technology, Czech Technical University in Prague, Thákurova 9, 16000 Prague, Czechia

<sup>2</sup> Department of Computing Science, Umeå University, Universitetstorget 4, 901 87 Umeå, Sweden

for union, intersection, complement, product, iteration, and reverse of regular languages. The outputs of the algorithm created as an FA and their positions during the reading of the input string are typically represented by the final states of the FA. Studying this formalism and its properties can therefore lead to improving a large number of algorithms, as well as help creating new ones.

A strictly locally  $k$ -testable language (SLTL),  $k \geq 1$ , introduced in [6] and further studied in [7–9], is a language characterized by its factors of length  $k$  such that a string of length at least  $k$  is in the language if its prefix of length  $k - 1$ , suffix of length  $k - 1$  and each factor of length  $k$  belong to a set of allowed prefixes, of allowed suffixes and of allowed factors, respectively. Equivalently, one can test membership using sets of forbidden internal factors of length  $k$  and forbidden prefixes and suffixes of length  $k - 1$ . We note that in the definition there are no restrictions for strings shorter than  $k$ , therefore, to fully define the language we also need a set of either forbidden or allowed words shorter than  $k$ . For SLTL, since the length of the shortest such factors is bounded by  $k$ , the sets defining the language are finite. The class of SLTLs is accepted by local DFAs, where each string of length at least  $k - 1$  is synchronizing. The  $k$  is the smallest possible if the DFA is minimal.

A problem of extraction of the shortest forbidden factors, i.e., given a DFA finding the set of the shortest factors of a specific type (be it prefix, suffix, or inner factor) that guarantee non-acceptance when they are found in the input string, has been studied in the past and has its practical use-cases. Béal et al. [10] describe a quadratic-time algorithm to compute the set of minimal forbidden words of a factorial regular language. Rogers and Lambert [11, 12] present algorithms for extracting the shortest forbidden factors and suffixes from a local DFA, which arose from a practical need in their research of analyzing stress patterns in human languages. They also mention the possibility of extracting the shortest forbidden prefixes by extracting suffixes from a DFA accepting reversed language.

Our improvement is based on the approach of Rogers and Lambert. Their algorithm works by constructing a powerset graph of the input DFA (i.e., a directed labeled graph where nodes are subsets of states and edges labeled with input symbols lead to a node containing destination states of any transition from a source state on the symbol) and looking for specific paths in the graph which correspond to the resulting strings. This is done by traversing the graph from a specific state against the direction of edges. Since there can be multiple edges with the same symbol leading to the same state, each string can have multiple paths in such traversal. The traversal is therefore designed to go through all of these paths while also filtering them on the go to ensure we only get the shortest factors. A prerequisite of the traversal is also that the powerset graph is acyclic (on nodes of size at least 2), which is the case if and only if the DFA is local and as a result the shortest forbidden factors are finite.

We, however, show that no special traversal is needed if we modify the powerset construction in the right way. Our modification causes each string to have only one path, therefore we effectively create a DFA and its language is exactly the resulting set we want, or more precisely its reverse. A standard BFS or DFS is then sufficient to obtain the result if it is finite. With our approach, we can in a very straightforward way create a DFA construction from which we can create DFAs for each type of shortest characteristic factors just by choosing the correct initial and final states. This not only simplifies the existing extraction of the shortest forbidden factors and suffixes but also allows the direct extraction of more types of characteristic factors, specifically shortest forbidden prefixes and words, and shortest allowed suffixes and words.

Another advantage is that our extraction works for any DFA, not just local ones. If the shortest characteristic factors are finite then the DFAs are acyclic. Otherwise, they describe an infinite language. As mentioned, the language is the reverse of the result. If an unreversed

result is needed, it is straightforward for DFAs with finite languages; otherwise, the standard construction of a DFA for the reversed language would be necessary.

Furthermore, we describe that extraction of the shortest characteristic factors can also be used to achieve a sublinear time of some FA-based algorithms by utilizing exact string pattern set matching. Pattern matching is a fundamental and widely studied problem [13, 14]. Given a text and patterns, the pattern matching searches for all factors of the text that match the patterns. Among the most efficient pattern-matching algorithms in practice are those based on backward pattern matching [15, 16]. Although the time complexity of backward string pattern matching is generally  $\mathcal{O}(n * m)$  in the worst case (for a text and a pattern of size  $n$  and  $m$ , respectively), due to the ability of the backward string pattern matching algorithms to skip parts of the text, they often perform sublinearly in practice [17, 18].

We define a positive position run (PPR) of a deterministic finite automaton, which represents all positions in an input string in which the automaton reaches a final state. This notion commonly corresponds to the output of FA-based algorithms. Certain string pattern matching problems can for example be solved by FAs where positions of final states represent the occurrences of matches in the input string [5, 19]. We then show the computation of PPR by matching sets of the shortest forbidden factors and the shortest allowed suffixes, if they are finite. The running time is then inherited from the pattern-matching algorithm used, therefore with a backward pattern-matching algorithm it can be sublinear for certain inputs. We demonstrate this using the Commentz-Walter algorithm. We are not aware of any algorithms computing runs of DFAs using pattern matching.

This paper is a full version of [20]. This version provides full proofs and also adds constructions of DFAs for the shortest forbidden and allowed words.

## 2 Basic notions

An *alphabet* is a finite set of *symbols*. A *string* (or a *word*)  $w$  over an alphabet  $\Sigma$  is a sequence  $w = a_1 a_2 \dots a_n$  such that  $a_i \in \Sigma$ ,  $n \geq i \geq 1$ .  $w[i]$  denotes the  $i$ -th symbol of  $w$ ,  $|w|$  is the length of  $w$ ,  $w^R$  is the reverse of  $w$ ,  $\varepsilon$  denotes the string of length 0 and  $\Sigma^*$  denotes the set of all string over  $\Sigma$ . A *language* over an alphabet  $\Sigma$  is a set  $L \subseteq \Sigma^*$  and  $L^R$  denotes the reverse of the language. A *prefix* and a *suffix* of  $w$  is a string  $u$  such that  $w = uv$  and  $w = vu$ , respectively, for some  $v \in \Sigma^*$ . The prefix and the suffix is *proper* if  $|u| < |w|$ . A factor of  $w$  is a string  $u$  such that  $w = tuv$  for some  $t, u, v \in \Sigma^*$  and it is *proper* if  $|u| < |w|$ .

A (partial) *deterministic finite automaton* (DFA) over an alphabet  $\Sigma$  is a 5-tuple  $A = (Q, \Sigma, \delta, q_0, Q_f)$ , where  $Q$  is a finite set of states,  $\delta: Q \times \Sigma \rightarrow Q$  is a partial transition function,  $q_0 \in Q$  is an initial state, and  $Q_f \subseteq Q$  is a set of final states. The DFA is *complete* (or *total*) if  $\delta$  is total. An *extended transition function*  $\delta^*: Q \times \Sigma^* \rightarrow Q$  is a partial function such that for each  $q \in Q: \delta^*(q, \varepsilon) = q$  and if  $\delta(q, a)$  and  $\delta^*(\delta(q, a), w)$  exist, then  $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$ , where  $a \in \Sigma$  and  $w \in \Sigma^*$ . Otherwise,  $\delta^*(q, aw)$  is undefined. A DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is said to *accept* a string  $w \in \Sigma^*$  if  $\delta^*(q_0, w) \in Q_f$ . A *language of a state*  $q \in Q$  is  $L(q) = \{w \in \Sigma^* \mid \delta^*(q, w) \in Q_f\}$ . A *language accepted by the DFA*  $A$  is  $L(A) = L(q_0)$ . A state  $q \in Q$  is *reachable* if  $\delta^*(q_0, w) = q$  for some  $w \in \Sigma^*$ . In this paper we consider only DFAs without unreachable states. The DFAA is *minimal* if all states are reachable and  $\forall q_1, q_2 \in Q: L(q_1) \neq L(q_2)$ . The DFAA is *complete* if  $\delta$  is complete. A *useless state* for  $A$  is any state  $q \in Q$  such that  $L(q) = \emptyset$ . A minimal complete DFA has at most one useless state. For  $A$ ,  $A^R$  is a DFA such that  $L(A^R) = L(A)^R$ . A *run* of

the DFA on string  $w \in \Sigma^*$  is a sequence  $q_0, \dots, q_{|w|}$  such that  $q_i = \delta^*(q_0, w_i)$ , where  $w_i$  is a prefix of  $w$  and  $|w_i| = i$ .

For a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  a string  $w$  is *synchronizing* to a set of states  $S \subseteq Q$  if  $\forall q \in Q : \delta^*(q, w) \in S$  if  $\delta^*(q, w)$  exists. The DFA is *k-local* if for each string  $w$  of length at least  $k$ , there exists some state  $q$  for which  $w$  is synchronizing to  $q$ . If the DFA is *k-local* for some  $k$  then it is said to be *local*.

**Definition 1** ((Shortest) forbidden factor) A *forbidden factor* of a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\forall q \in Q : \delta^*(q, w)$  is undefined or  $L(\delta^*(q, w)) = \emptyset$ .  $\text{FF}(A)$  is the set of all forbidden factors of  $A$  and  $w \in \text{FF}(A)$  is a *shortest forbidden factor* iff  $u \notin \text{FF}(A)$  for all proper factors  $u$  of  $w$ .  $\text{SFF}(A) \subseteq \text{FF}(A)$  is the set of all the shortest forbidden factors of  $A$ .

In other words, the forbidden factors will cause the DFA to fail by a missing transition or ending in a useless state, regardless of the starting conditions.

**Definition 2** ((Shortest) forbidden suffix) A *forbidden suffix* of a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\forall q \in Q : \delta^*(q, w) \notin Q_f$  or  $\delta^*(q, w)$  is undefined.  $\text{FS}(A)$  is the set of all forbidden suffixes of  $A$  and  $w$  is a *shortest forbidden suffix* iff  $w \notin \text{FF}(A)$  and  $u \notin \text{FS}(A)$  for all proper suffixes  $u$  of  $w$ .  $\text{SFS}(A) \subseteq \text{FS}(A)$  is the set of all the shortest forbidden suffixes of  $A$ .

In other words, the forbidden suffixes will cause the DFA to either end in a non-final state or fail by a missing transition or ending in a useless state, regardless of the starting conditions. The set of shortest forbidden suffixes does not contain forbidden factors, as such information is redundant, as explained later.

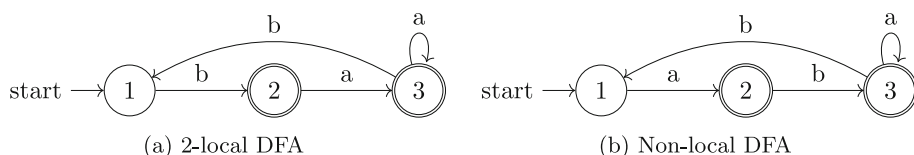
**Definition 3** ((Shortest) allowed suffix) An *allowed suffix* of a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\forall q \in Q : \delta^*(q, w) \in Q_f$  or  $\delta^*(q, w)$  is undefined or  $L(\delta^*(q, w)) = \emptyset$ , and also  $\delta^*(q', w) \in Q_f$  for at least one  $q' \in Q$ .  $\text{AS}(A)$  is the set of all allowed suffixes of  $A$  and  $w$  is a *shortest allowed suffix* iff  $u \notin \text{AS}(A)$  for all proper suffixes  $u$  of  $w$ .  $\text{SAS}(A) \subseteq \text{AS}(A)$  is the set of all the shortest allowed suffixes of  $A$ .

In other words, the allowed suffixes will cause the DFA to either end in a final state or fail by a missing transition or ending in a useless state, regardless of the starting conditions. At least one such path to a final state should exist; otherwise it would be a forbidden factor. Essentially, having an allowed suffix guarantees acceptance of the string, provided that the automaton was able to read the entire string.

**Definition 4** ((Shortest) forbidden prefix) A *forbidden prefix* of a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\delta^*(q_0, w)$  is undefined or  $L(\delta^*(q_0, w)) = \emptyset$ .  $\text{FP}(A)$  is the set of all forbidden prefixes of  $A$  and  $w$  is a *shortest forbidden prefix* iff  $w \notin \text{FF}(A)$  and  $u \notin \text{FP}(A)$  for all proper prefixes  $u$  of  $w$ .  $\text{SFP}(A) \subseteq \text{FP}(A)$  is the set of all the shortest forbidden prefixes of  $A$ .

In other words, the forbidden prefixes are the same concept as forbidden factors, but we only consider starting in the initial state. Same as forbidden suffixes, the set of shortest forbidden prefixes does not contain forbidden factors, as such information is redundant, as explained later.

**Definition 5** (Shortest forbidden word) A *shortest forbidden word* of a DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in Q \setminus Q_f$ ,  $L(\delta^*(q_0, w)) \neq \emptyset$  and  $w \notin \text{FS}(A)$ .  $\text{SFW}(A) \subseteq \overline{L(A)}$  is the set of all the shortest forbidden words of  $A$ .



**Fig. 1** Example of a local and a non-local partial DFA

The shortest forbidden words are all strings that do not belong in the language of the DFA, but at the same time do not contain any occurrence of other types of forbidden factors. By requiring to reach a useful state from the initial, we filter out forbidden prefixes and factors. From forbidden factors, they are then differentiated by whether it is possible to reach a final state from some starting state.

**Definition 6** (*Shortest allowed word*) A *shortest allowed word* of a DFAA  $= (Q, \Sigma, \delta, q_0, Q_f)$  is a string  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in Q_f$  and  $w \notin AS(A)$ .  $SAW(A) \subseteq L(A)$  is the set of all the shortest allowed words of  $A$ .

The shortest allowed words are all strings that belong to the language of the DFA, but at the same time do not contain any occurrence of an allowed factor.

Essentially, both shortest forbidden and allowed words contain strings, where a membership in the language cannot be decided just by analyzing its factors.

If a DFAA is local then all sets  $SFF(A)$ ,  $SFS(A)$ ,  $SFP(A)$ ,  $SFW(A)$ ,  $SAS(A)$  and  $SAW(A)$  are finite and  $L(A)$  is strictly locally testable. We call these sets the *shortest characteristic factors* of  $A$ .

As an example, the 2-local automaton in Fig. 1a has the following characteristic factors:  $SFF(A) = \{aba, bbb\}$ ,  $SFP(A) = \{a, bb\}$ ,  $SFS(A) = \{ab\}$ ,  $SFW(A) = \{\varepsilon\}$ ,  $SAS(A) = \{a, bb\}$ ,  $SAW(A) = \{b\}$ . The non-local automaton in Fig. 1b has the following characteristic factors:  $SFF = \{aab(abab)^*aa, bb(abab)^*aa\}$ ,  $SFP = \{(abab)^*aa, (abab)^*b, \}$ ,  $SFS = \{aa(baba)^*b, b(baba)^*b\}$ ,  $SFW = \{(abab)^*\}$ ,  $SAS = \{a, aab(abab)^*ab, bb(abab)^*ab\}$ ,  $SAW = \{(abab)^*ab\}$ , where  $(\dots)^*$  means repeated zero or more times, implying that the set is infinite.

Based on previous definitions, we can observe the following properties.

**Remark 1** For a DFA  $A$  the following statements hold:

$$\begin{aligned}
 w \in FF(A) &\iff w \text{ contains some factor in } FF(A) \\
 &\iff w \text{ contains some factor in } SFF(A) \\
 w \in FP(A) &\iff w \text{ contains some prefix in } FP(A) \\
 &\iff w \text{ contains some prefix in } SFP(A) \text{ or } w \in FF(A) \\
 w \in FS(A) &\iff w \text{ contains some suffix in } FS(A) \\
 &\iff w \text{ contains some suffix in } SFS(A) \text{ or } w \in FF(A) \\
 w \in AS(A) &\iff w \text{ contains some suffix in } AS(A) \text{ and } w \notin FF(A) \\
 &\iff w \text{ contains some suffix in } SAS(A) \text{ and } w \notin FF(A)
 \end{aligned}$$

Given these shortest characteristic factors, the language of any DFA  $A$  (and therefore any regular language) can be fully described in the following way:

**Proposition 1** Let us have a DFA  $A = (Q, \Sigma, \delta, q_0, Q_f)$ . For some  $w \in \Sigma^*$  we have  $w \in L(A)$  iff the following conditions are met:

- 1  $w \notin \text{SFW}(A)$ ,
- 2  $w' \notin \text{SFP}(A)$  for all prefixes  $w'$  of  $w$ ,
- 3  $w' \notin \text{SFF}(A)$  for all factors  $w'$  of  $w$ ,
- 4  $w' \notin \text{SFS}(A)$  for all suffixes  $w'$  of  $w$ .

Allowed words and suffixes can be used instead of the forbidden ones. The following statement is equivalent:

- $w \in L(A)$  iff  $w \in \text{SAW}(A)$ , or  $w' \in \text{SAS}(A)$  for some suffix  $w'$  of  $w$  and conditions 2 and 3.

**Proof** If  $w \in L(A)$  then  $\delta^*(q_0, w)$  is a final state. If  $w \notin L(A)$ , we consider two disjoint cases. The first one is that  $\delta^*(q_0, w)$  is a useful non-final state, and the second one is that  $\delta^*(q_0, w)$  is either undefined or a useless state.

Let us start by considering that  $\delta^*(q_0, w)$  is undefined or a useless state. Then, by definition,  $w \in \text{FP}(A)$  and, based on Remark 1, it is equivalent to  $w$  containing a prefix from  $\text{SFP}(A)$  or a factor from  $\text{SFF}(A)$ , which happens iff conditions 2 or 3 are not satisfied. In this case, also by definition,  $w \notin \text{SAW}(A)$ .

For the next case, let us assume  $\delta^*(q_0, w) \in Q \setminus Q_f$  and the resulting state is useful. From the previous case, we can infer that conditions 2 and 3 are satisfied. From definitions, we can infer that  $w \in \text{SFW}(A)$  if there exists some  $q' \in Q$  for which  $\delta^*(q', w) \in Q_f$ , otherwise  $w \in \text{FS}(A)$ , and in such case, based on Remark 1,  $w$  either contains a suffix in  $\text{SFS}(A)$ , violating condition 4, or  $w \in \text{FF}(A)$ , which cannot happen since the condition 3 is satisfied. Regarding the allowed sets, we can trivially infer that  $w \notin \text{SAW}(A)$ , and also that  $w$  does not contain any suffix in  $\text{SAS}(A)$  since for any suffix  $w'$  of  $w$  there exists some  $q'' \in Q$ , such that  $\delta^*(q'', w') = \delta^*(q_0, w) \in Q \setminus Q_f$ .

In the last case, when  $\delta^*(q_0, w) \in Q_f$ , conditions 2 and 3 again has to hold, since the result is defined and a useful state. The condition 1 trivially has to hold, and in case  $w$  contains any occurrence of suffixes from  $\text{SFS}(A)$ , based on Remark 1, we would have  $w \in \text{FS}(A)$ , indicating that  $\delta^*(q_0, w) \notin Q_f$ . Therefore, condition 4 also has to hold. Alternatively, based on definitions, we have  $w \in \text{SAS}(A)$ , if there exists some useful state  $q' \in Q$  for which  $\delta^*(q', w) \in Q \setminus Q_f$ , otherwise  $w \in \text{AS}(A)$ , and in such case, based on Remark 1,  $w \notin \text{FF}(A)$ , which does hold here, and  $w$  has to contain a suffix in  $\text{SAS}(A)$ . Therefore, we either have  $w \in \text{SAW}(A)$ , or  $w$  contains a suffix in  $\text{SAS}(A)$  and conditions 2 and 3 are satisfied.  $\square$

As illustrated by the Proposition 1, the shortest characteristic factors can be viewed as the minimal sets of strings which can be utilized to decide a membership of a string in a language. We note that the shortest forbidden prefixes and suffixes are disjoint with the shortest forbidden factors since for characterization of the language by Proposition 1 such prefix or suffix would be redundant. For the same reason the shortest forbidden words are disjoint with the shortest forbidden prefixes and suffixes and the shortest allowed words are disjoint with the shortest allowed suffixes.

We also note that allowed suffixes are strings that starting from any state always either end in a final state or fail (either by a missing transition or reaching a useless state). If however they only fail, then it is a forbidden factor instead, therefore at least one path needs to end in a final state.

**Lemma 1** (Relationship of forbidden prefixes and suffixes [11, 12]) *For a minimal DFAA it applies that  $\text{FS}(A) = \text{FP}(A^R)$  and  $\text{SFS}(A) = \text{SFP}(A^R)$ .*

In Sect. 5 we introduce a pattern set matching approach to computing a version of an automaton run. As a consequence, the algorithm inherits properties of the pattern matching algorithm used. We demonstrate it using Commentz-Walter algorithm, which is a backwards pattern set matching algorithm that can skip over parts of the input string, which allows it to run in sublinear time for certain inputs and experiments indicate this is the case on average. The reversed pattern set is preprocessed into a trie structure with some additional information in each vertex, like functions  $\text{shift1}$  and  $\text{shift2}$ , which are used by the algorithm to determine how far to shift in the input after a mismatch. We however strongly emphasize that any pattern set matching algorithm can be used and the choice is important only for the properties. For this reason, we do not describe the Commentz-Walter and all its notions in detail. For that see [16] and some relevant details can also be found in Appendix A. We consider a slight modification that matching certain patterns will cause the algorithm to terminate.

### 3 Synchronization properties of characteristic factors

We first describe some properties of synchronizing strings and characteristic factors, which will serve as the basis of our constructions. The following lemmas describe how such strings behave in a DFA with a complete transition function, which our construction will rely upon. If the automaton's transition function is not complete, we can complete it by adding a new special useless state. Instead of all missing transitions, we add transitions to this useless state. We can equivalently state the synchronization property of a DFA after completing the transition function in the following way.

**Lemma 2** *Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a DFA without useless states and  $A' = (Q \cup U, \Sigma, \delta', q_0, Q_f)$  be a complete DFA such that  $U$  is a set of useless states,  $\delta \subseteq \delta'$  and  $L(A) = L(A')$ . A string  $w \in \Sigma^*$  is synchronizing to a set of states  $S \subseteq Q$  in  $A$  if and only if it is synchronizing to  $S \cup U$  in  $A'$ .*

**Proof** For all  $q \in Q$  if  $\delta^*(q, w)$  exists then  $\delta^*(q, w) = \delta'^*(q, w)$  since  $\delta \subseteq \delta'$ , otherwise  $\delta'^*(q, w) \in U$ . Also for all  $u \in U$  :  $\delta'^*(u, w) \in U$ . Therefore, if  $w$  is synchronizing to  $S \subseteq Q$  in  $A$  then it is also synchronizing to  $S \cup U$  in  $A'$ .

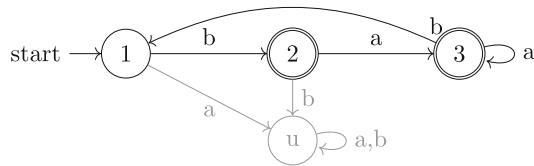
For the other way around, if  $w$  is synchronizing to  $S' \subseteq Q \cup U$  in  $A'$  and  $U \neq \emptyset$ , then  $S' \cap U \neq \emptyset$  since for all  $u \in U$  :  $\delta'^*(u, w') \in U$  for any string  $w' \in \Sigma^*$ . For all  $q \in Q \cup U$  if  $\delta'^*(q, w) \in U$ , then  $\delta^*(q, w)$  does not exist, otherwise  $\delta'^*(q, w) = \delta^*(q, w)$ . Therefore,  $w$  is synchronizing to  $S' \setminus U = S$  in  $A$ .  $\square$

Consequently, the following corollaries hold.

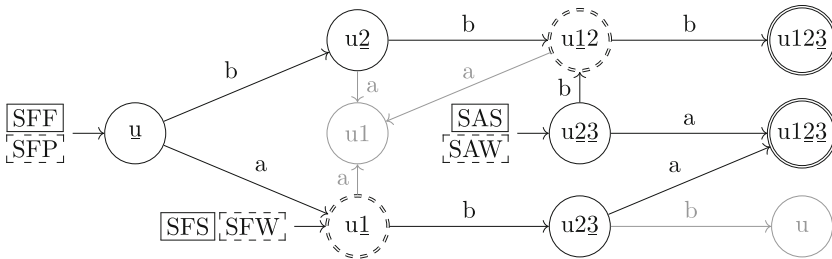
**Corollary 1** (Forbidden Factors) *Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with  $U \subseteq Q$  being a set of useless states. A string  $w \in \Sigma^*$  is a forbidden factor of  $A$  if and only if  $w$  is synchronizing to  $U$  in  $A$ .*

**Corollary 2** (Forbidden Suffixes) *Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA. A string  $w \in \Sigma^*$  is a forbidden suffix of  $A$  if and only if  $w$  is synchronizing to  $Q \setminus Q_f$ .*

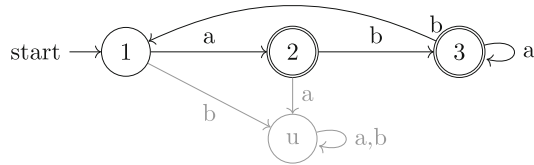
**Corollary 3** (Allowed Suffixes) *Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with  $U \subseteq Q$  being a set of useless states. A string  $w \in \Sigma^*$  is an allowed suffix of  $A$  if and only if  $w$  is synchronizing to  $Q_f \cup U$  in  $A$  and there exists a state  $q \in Q$  such that  $\delta^*(q, w) \in Q_f$ .*



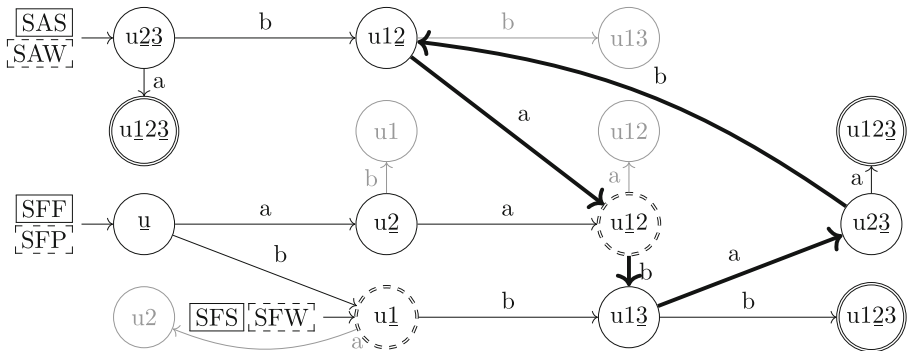
(a) 2-local DFA



(b) DFA for the shortest characteristic factors of the DFA in (a)



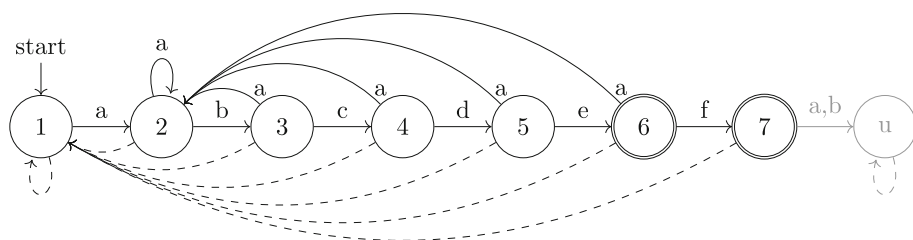
(c) Non-local DFA



(d) DFA for the shortest characteristic factors of the DFA in (c); Thick transitions show a cycle caused by non-locality of the original DFA

**Fig. 2** Examples of automata for the shortest characteristic factors **(b)** and **(d)** of a total local and a non-local DFAs **(a)** and **(c)**. Gray states are useless. Automata in **(b)** and **(d)** show initial state for each type of characteristic factors, and the rectangle type in the label indicates its corresponding final states (solid or dashed)





**Fig. 3** A 6-local partial DFAA over alphabet  $\{a, b, c, d, e, f\}$  with an added useless state  $u$ . Dashed transitions are for all symbols without explicit transition.  $\text{SFF}(A) = \{abcdefa, abcdefb\}$ ,  $\text{SAS}(A) = \{abcde, abcdef\}$  and  $\text{SFS}(A)$  contains 49 strings

In case of forbidden prefixes and shortest forbidden and allowed words, they are characterized mainly by paths from the initial state and as such they don't have to be synchronizing. And in case of the shortest ones they in fact cannot be synchronizing because then they would be either the shortest forbidden factor, suffix or allowed suffix instead.

**Remark 2** (Forbidden Prefixes) Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with  $U \subseteq Q$  being a set of useless states. A string  $w \in \Sigma^*$  is a forbidden prefix of  $A$  if and only if  $\delta^*(q_0, w) \in U$ .

For shortest forbidden and allowed words there is no change since in their case a path from the initial state must always reach some useful state.

**Remark 3** A complete DFA without useless states has no forbidden factors or prefixes.

## 4 Automata accepting shortest characteristic factors

In this section we describe the construction of automata accepting the shortest characteristic factors of an input DFA. We start with an informal description of the construction, and then we proceed to formally defining it and proving the language accepted by each of the constructed DFAs.

### 4.1 Construction overview

The construction is a variant of the powerset graph construction of Rogers et al. [11, 12]. The main difference is that original states in a subset will also have a boolean mark associated with them and edges will be constructed based on incoming transition to the original states rather than outgoing transitions, i.e., against the direction of original transitions.

Based on this underlying graph we will then be able to construct DFAs for each type of characteristic factor just by choosing the right initial and final states. The language of the DFA will be the reverse of the set of the shortest characteristic factors of the specific type. If the set is finite, which is guaranteed for local DFAs, the constructed DFA will be acyclic (on reachable useful states). Otherwise, the DFA will accept an infinite language, which is still correct under the definition of the shortest characteristic factors.

For an input DFA the constructed automata contain the following parts:

- states:

**Table 1** Initial and final states combinations for shortest characteristic factors DFAs

| Initial          |          | Final                            |                                                      |
|------------------|----------|----------------------------------|------------------------------------------------------|
| Marked           | Unmarked | All states,<br>at least 1 marked | Any states (but not all),<br>at least initial marked |
| Useless          | –        | SFF                              | SFP                                                  |
| Useful non-final | Useless  | SFS                              | SFW                                                  |
| Final            | Useless  | SAS                              | SAW                                                  |

- subset of input DFA states with a boolean mark associated to each input state.
- transitions:
  - for each input DFA state in the source of the transition the destination contains all sources of incoming transitions (on the same symbol) to that state.
  - an original state in the destination is marked if the state is not useless and some of its transitions (on the same symbol) lead to a marked state in the source.
  - the transition is undefined if the source contains all original states.
- initial and final states:
  - there are three types of initial and two types of final states, their combination characterizes DFA for a specific characteristic factor type (see Table 1).
  - initial state:
    - State containing marked all useless input DFA states (for SFF and SFP).
    - State containing marked all useful non-final input DFA states and unmarked useless states (for SFS and SFW).
    - State containing marked all final input DFA states and unmarked useless states (for SAS and SAW).
  - final states:
    - States containing all input DFA states where at least one of them is marked (for SFF, SFS and SAS)
    - States containing any (but not all) input DFA states where at least the initial state is marked (for SFP, SFW and SAW)

Figure 2 shows an example of this construction for a local and a non-local input DFA. Also, we note that any state not having a marked original state is always useless.

The mark is used to indicate a very simple property that turns out to be sufficient to characterize each of the shortest characteristic factor types. Every path in the constructed DFA from some source to some destination indicates that in the input DFA there exists a path on the same string between some marked destination state to some marked source state that does not contain a transition between useless states. In case of, e.g., shortest forbidden factors it tells us that there exists a path from some state reaches a useless state only at its end and therefore any of its prefixes is no longer a forbidden factor. In case of, e.g., shortest allowed suffixes it helps us to ensure that there is at least one path that ends in a final state and not in just useless states.

## 4.2 Formal description

We first start by defining the basic structure of the constructed DFAs, which we call shortest characteristic factors template automaton. Based on the shortest characteristic factors template automaton, we then define automata accepting the shortest characteristic factors, and then we prove the correctness of languages they accept.

**Definition 7** (*Shortest characteristic factors template automaton*)  $\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f) = (\widehat{Q}, \Sigma, \widehat{\delta}, \widehat{q}_0, \widehat{Q}_f)$  for some complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$  and some  $\widehat{q}_0 \in \widehat{Q}$  and  $\widehat{Q}_f \subseteq \widehat{Q}$  such that:

- $\widehat{Q} = \{\widehat{q} \mid \widehat{q}: Q' \rightarrow \{0, 1\}, Q' \subseteq Q\}$
- For  $S \in \widehat{Q} : \text{dom}(S) = \{q \mid (q, x) \in S, x \in \{0, 1\}\}$
- $\widehat{\delta}$  is a partial function  $\widehat{Q} \times \Sigma \rightarrow \widehat{Q}$  such that if  $\text{dom}(S) \neq Q$  then  $\widehat{\delta}(S, a) = \{(q, x) \mid \delta(q, a) \in \text{dom}(S)\}$ , where  $x = ((\delta(q, a), 1) \in S \wedge q \notin U)$

**Definition 8** For a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$  we define the following states, sets of states and automata:

- $\widehat{q}_U = U \times \{1\}$
- $\widehat{q}_f = (U \times \{0\}) \cup (Q_f \times \{1\})$
- $\widehat{q}_{nf} = (U \times \{0\}) \cup (Q \setminus (Q_f \cup U) \times \{1\})$
- $\widehat{Q}_Q = \{S \mid S \in \widehat{Q} \wedge \text{dom}(S) = Q \wedge (\exists q \in Q : (q, 1) \in S)\}$
- $\widehat{Q}_0 = \{S \mid S \in \widehat{Q} \wedge \text{dom}(S) \neq Q \wedge (q_0, 1) \in S\}$
- $\widehat{A}_{\text{SFF}}(A) = \widehat{A}(A, \widehat{q}_U, \widehat{Q}_Q)$
- $\widehat{A}_{\text{SFP}}(A) = \widehat{A}(A, \widehat{q}_U, \widehat{Q}_0)$
- $\widehat{A}_{\text{SFS}}(A) = \widehat{A}(A, \widehat{q}_{nf}, \widehat{Q}_Q)$
- $\widehat{A}_{\text{SFW}}(A) = \widehat{A}(A, \widehat{q}_{nf}, \widehat{Q}_0)$
- $\widehat{A}_{\text{SAS}}(A) = \widehat{A}(A, \widehat{q}_f, \widehat{Q}_Q)$
- $\widehat{A}_{\text{SAW}}(A) = \widehat{A}(A, \widehat{q}_f, \widehat{Q}_0)$

Algorithm 1 shows the construction of such automata. We note that if within a state of the new DFA there are no original states with flags set to true, then such a state is trivially useless, since transition from such a state will never set a flag to true, which is necessary in both types of final states. In our examples and in Algorithm 1 we therefore omit further transitions from such states.

We will now proceed to prove the languages of the constructed automata.

**Lemma 3** Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with a set of useless states  $U \subseteq Q$  and  $\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f) = (\widehat{Q}, \Sigma, \widehat{\delta}, \widehat{q}_0, \widehat{Q}_f)$  for some  $\widehat{q}_0 \in \widehat{Q}$  and  $\widehat{Q}_f \subseteq \widehat{Q}$ . Then  $\forall w \in \Sigma^*, S \in \widehat{Q} : \widehat{\delta}^*(\widehat{q}_0, w) = S$  iff:

1.  $Q \neq \text{dom}(\widehat{\delta}(\widehat{q}_0, u))$  for all proper prefixes  $u$  of  $w$
2.  $\text{dom}(S) = \{q \mid \delta^*(q, w^R) \in \text{dom}(\widehat{q}_0)\}$
3.  $\forall (q, x) \in S : x = ((\delta^*(q, w^R), 1) \in \widehat{q}_0 \wedge (w = a'v \implies \delta^*(q, v^R) \notin U))$  for all  $a' \in \Sigma, v \in \Sigma^*$

**Proof** We prove both implications inductively, starting with the left-to-right. For  $w = \varepsilon$ , since  $\widehat{\delta}(\widehat{q}_0, \varepsilon) = \widehat{q}_0$ , all conditions trivially apply. For  $w = w'a$ , where  $w' \in \Sigma^*$  and  $a \in \Sigma$ , let  $S' = \widehat{\delta}^*(\widehat{q}_0, w')$ .

**Algorithm 1:** Construction of automaton  $\hat{A}$ 


---

**Require:** Complete DFAA  $= (Q, \Sigma, \delta, q_0, Q_f)$  with useless states  $U \subseteq Q$ , initial states  $\hat{Q}_i \subseteq \{\hat{q}_0 \subseteq \{Q \rightarrow \{0, 1\}\}\}$

**Ensure :** Not trivially useless parts of  $\hat{Q}$ ,  $\hat{\delta}$ ,  $\hat{Q}_Q$  and  $\hat{Q}_0$  reachable from some  $\hat{q}_0$

```

1  $\hat{Q} \leftarrow \hat{Q}_i$ 
2 Let  $qu$  be a queue
3 foreach  $\hat{q}_0 \in \hat{Q}_i : \exists q \in Q : \hat{q}_0(q) = 1$  do  $qu.enqueue(\hat{q}_0)$  while  $qu$  not empty do
4    $\hat{q} \leftarrow qu.dequeue()$ 
5   if  $|\hat{q}| = |Q|$  then
6      $\hat{Q}_Q \leftarrow \hat{Q}_Q \cup \hat{q}$ 
7     continue
8   if  $\hat{q}(q_0) = 1$  then  $\hat{Q}_0 \leftarrow \hat{Q}_0 \cup \hat{q}$  foreach  $a \in \Sigma$  do
9      $\hat{q}' \leftarrow \emptyset$ 
10    foreach  $q \in Q : \delta(q, a) \in \text{dom}(\hat{q})$  do  $\hat{q}'(q) \leftarrow \hat{q}(\delta(q, a)) \wedge q \notin U$  if  $\exists q \in Q : \hat{q}'(q) = 1$  then
11       $\hat{\delta}(\hat{q}, a) \leftarrow \hat{q}'$ 
12      if  $\hat{q}' \notin \hat{Q}$  then
13         $\hat{Q} \leftarrow \hat{Q} \cup \hat{q}'$ 
14         $qu.enqueue(\hat{q}')$ 

```

---

1. If  $S'$  exists then  $Q \neq \text{dom}(\hat{\delta}(\hat{q}_0, u'))$  for all proper prefixes  $u'$  of  $w'$ . If  $Q \neq \text{dom}(S')$  then  $Q \neq \text{dom}(\hat{\delta}(\hat{q}_0, w'))$  as well. Otherwise, if  $Q = \text{dom}(S')$  then by definition  $\hat{\delta}(S', a)$  does not exist. In such case or if  $S'$  does not exist then  $\hat{\delta}^*(\hat{q}_0, w'a)$  does not exist either, which contradicts that  $\hat{\delta}^*(\hat{q}_0, w) = S$ .
2.  $\text{dom}(S) = \{q \mid \delta(q, a) \in \text{dom}(S')\}$   
 $= \{q \mid \delta^*(\delta(q, a), w'^R) \in \text{dom}(\hat{q}_0)\}$   
 $= \{q \mid \delta^*(q, aw'^R) \in \text{dom}(\hat{q}_0)\}$
3. Let  $v' \in \Sigma^*$ . For each  $(q, x) \in S$  applies:

$$\begin{aligned}
x &= (\delta(q, a), 1) \in S' \wedge q \notin U \\
&= (\delta^*(\delta(q, a), w'^R), 1) \in \hat{q}_0 \wedge (w' = a'v' \implies \delta^*(\delta(q, a), v'^R) \notin U) \wedge q \notin U \\
&= (\delta^*(q, aw'^R), 1) \in \hat{q}_0 \wedge \left( w = a'v \implies \begin{cases} q \notin U & v = \varepsilon \\ \delta^*(q, av'^R) \notin U & v = v'a \end{cases} \right) \\
&= (\delta^*(q, w^R), 1) \in \hat{q}_0 \wedge (w = a'v \implies \delta^*(q, v^R) \notin U)
\end{aligned}$$

For the right-to-left implication for  $w = \varepsilon$  all conditions trivially imply that  $S = \hat{q}_0$ . For  $w = w'a$  where  $w' \in \Sigma^*$ ,  $a \in \Sigma$  let  $S' = \hat{\delta}^*(\hat{q}_0, w')$ .

1. If  $Q \neq \text{dom}(\hat{\delta}^*(\hat{q}_0, u))$  holds for all proper prefixes  $u$  of  $w$  then it also holds  $Q \neq \text{dom}(\hat{\delta}^*(\hat{q}_0, u'))$  and  $Q \neq \text{dom}(\hat{\delta}^*(\hat{q}_0, w'))$  for all proper prefixes  $u'$  of  $w'$ . That implies  $Q \neq \text{dom}(S')$ .
2. If  $\text{dom}(S) = \{q \mid \delta^*(q, aw'^R) \in \text{dom}(\hat{q}_0)\}$  then, by reversing equations in proof of the left-to-right implication, we obtain  $\text{dom}(S) = \{q \mid \delta(q, a) \in \text{dom}(S')\}$ .
3. Similarly, from the 3rd condition we obtain  $\forall (q, x) \in S : x = ((\delta(q, a), 1) \in S' \wedge q \notin U)$

All of these conditions by definition of  $\hat{\delta}$  imply that  $\hat{\delta}(S', a) = S$  which therefore means  $\hat{\delta}(\hat{\delta}^*(\hat{q}_0, w'), a) = \hat{\delta}^*(q_0, w'a) = S$ .  $\square$

The following lemma describes properties of languages of automata having  $\widehat{Q}_Q$  as final states, i.e.,  $\widehat{A}_{\text{SFF}}$ ,  $\widehat{A}_{\text{SFS}}$  and  $\widehat{A}_{\text{SAS}}$ .

**Lemma 4** Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with a set of useless states  $U \subseteq Q$ ,  $\widehat{Q}_f = \widehat{Q}_Q$  and  $\widehat{q}_0 = (Q_0 \times 0) \cup (Q_1 \times 1)$  for some  $Q_0, Q_1 \subseteq Q$ ,  $Q_0 \cap Q_1 = \emptyset$ .  $L(\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f))^R$  is a set of all strings  $w \in \Sigma^*$  such that  $w$  is a shortest string synchronizing to  $Q_0 \cup Q_1$  in  $A$  and there exists a state  $q \in Q$  such that  $\delta^*(q, w) \in Q_1$  and  $\delta^*(q, w') \notin U$  for all proper prefixes  $w'$  of  $w$ .

**Proof** Let  $\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f) = (\widehat{Q}, \Sigma, \widehat{\delta}, \widehat{q}_0, \widehat{Q}_f)$ . If a  $\widehat{q} \in \widehat{Q}_f$  then  $Q = \text{dom}(\widehat{q})$  and there exists at least one  $q \in Q$  such that  $(q, 1) \in \widehat{q}$ . Then Lemma 3 ensures that  $\widehat{\delta}(\widehat{q}_0, w^R) = \widehat{q}$  applies for all strings  $w$  satisfying that  $\forall q \in Q : \delta^*(q, w) \in Q_0 \cup Q_1$ , which implies that  $w$  is a synchronizing string to  $Q_0 \cup Q_1$ , and  $Q \neq \text{dom}(\widehat{\delta}(\widehat{q}_0, v^R))$  for all proper suffixes  $v$  of  $w$ , which implies  $w$  is the shortest synchronizing string since none of its proper suffixes synchronizes all states, and finally Lemma 3 also ensures that  $\delta^*(q, w) \in Q_1$  and if  $w = ua$  for some  $u \in \Sigma^*$  and  $a \in \Sigma$  then  $\delta^*(q, u) \notin U$ , which also implies  $\delta^*(q, w') \notin U$  for all proper prefixes  $w'$  of  $w$ .  $\square$

**Theorem 2**  $L(\widehat{A}_{\text{SFF}}(A))^R = \text{SFF}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** As stated in Corollary 1 forbidden factors of  $A$  are all strings  $w \in \Sigma^*$  synchronizing to  $U$  in  $A$ . For all such strings applies  $\delta^*(q, w) \in U$  for any state  $q \in Q$ . In order for  $w$  to be shortest it has to be the shortest synchronizing string and there has to be at least one state  $q \in Q$  such that  $\delta^*(q, w') \notin U$  for any proper prefix  $w'$  of  $w$ , otherwise  $w'$  would be a shorter forbidden factor. Therefore, using Lemma 4 and setting  $Q_0 = \emptyset$  and  $Q_1 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_U$ , we get that  $L(\widehat{A}(A, \widehat{q}_U, \widehat{Q}_Q))^R$  is the set of all the shortest forbidden factors of  $A$ .  $\square$

**Theorem 3**  $L(\widehat{A}_{\text{SAS}}(A))^R = \text{SAS}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** As stated in Corollary 3 allowed suffixes are strings  $w \in \Sigma^*$  synchronizing to  $Q_f \cup U$  in  $A$  for which there exists a state  $q \in Q$  such that  $\delta^*(q, w) \in Q_f$ . In such case  $\delta^*(q, w') \notin U$  for any proper prefix  $w'$  of  $w$  since a final state is unreachable from a useless state. Therefore, using Lemma 4 and setting  $Q_1 = Q_f$  and  $Q_2 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_f$ , we get that  $L(\widehat{A}(A, \widehat{q}_f, \widehat{Q}_Q))^R$  is the set of all the shortest allowed suffixes of  $A$ .  $\square$

**Theorem 4**  $L(\widehat{A}_{\text{SFS}}(A))^R = \text{SFS}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** As stated in Corollary 2 forbidden suffixes are strings  $w \in \Sigma^*$  synchronizing to  $Q \setminus Q_f$  in  $A$ . In order for the forbidden suffix to be the shortest, none of its proper suffixes can be forbidden and it cannot be a forbidden factor. Therefore,  $w$  needs to be a shortest synchronizing string to  $Q \setminus Q_f$  and there needs to be at least one state  $q \in Q$  such that  $\delta^*(q, w) \notin U$ , which consequentially implies  $\delta^*(q, w') \notin U$  for all prefixes of  $w$ . Therefore, using Lemma 4 and setting  $Q_1 = Q \setminus (Q_f \cup U)$  and  $Q_2 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_{nf}$ , we get that  $L(\widehat{A}(A, \widehat{q}_f, \widehat{Q}_Q))^R$  is the set of all the shortest forbidden suffixes of  $A$ .  $\square$

The following lemma describes properties of languages of automata having  $\widehat{Q}_0$  as final states, i.e.,  $\widehat{A}_{\text{SFP}}$ ,  $\widehat{A}_{\text{SFW}}$  and  $\widehat{A}_{\text{SAW}}$ .

**Lemma 5** Let  $A = (Q, \Sigma, \delta, q_0, Q_f)$  be a complete DFA with a set of useless states  $U \subseteq Q$ ,  $\widehat{Q}_f = \widehat{Q}_0$  and  $\widehat{q}_0 = (Q_0 \times 0) \cup (Q_1 \times 1)$  for some  $Q_0, Q_1 \subseteq Q$ ,  $Q_0 \cap Q_1 = \emptyset$ .  $L(\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f))^R$  is a set of all strings  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in Q_1$ ,  $w$  is not synchronizing to  $Q_0 \cup Q_1$  in  $A$  and  $\delta^*(q, w') \notin U$  for all proper prefixes  $w'$  of  $w$ .

**Proof** Let  $\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f) = (\widehat{Q}, \Sigma, \widehat{\delta}, \widehat{q}_0, \widehat{Q}_f)$ . If a  $\widehat{q} \in \widehat{Q}_f$  then  $Q \neq \text{dom}(\widehat{q})$  and  $(q_0, 1) \in \widehat{q}$ . Then Lemma 3 ensures that  $\widehat{\delta}(\widehat{q}_0, w^R) = \widehat{q}$  applies for all strings  $w$  satisfying that  $\text{dom}(\widehat{q}) = \{q \mid \delta^*(q, w) \in Q_0 \cup Q_1\}$  and  $\delta^*(q_0, w) \in Q_1$ . Since  $Q \neq \text{dom}(\widehat{q})$  then  $w$  is not synchronizing to  $Q_0 \cup Q_1$ .  $\square$

**Theorem 5**  $L(\widehat{A}_{\text{SFP}}(A))^R = \text{SFP}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** As stated in Remark 2, forbidden prefixes are strings  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in U$ . In order for the prefix to be shortest it has to apply that  $\delta^*(q_0, w') \in U$  for all proper prefixes  $w'$  of  $w$ , otherwise  $w'$  would be a shorter forbidden prefix, and  $w'$  cannot be synchronizing to  $U$ , otherwise it would be a forbidden factor. Therefore, using Lemma 5 and setting  $Q_0 = \emptyset$  and  $Q_1 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_U$ , we get that  $L(\widehat{A}(A, \widehat{q}_U, \widehat{Q}_0))^R$  is the set of all the shortest forbidden prefixes of  $A$ .  $\square$

**Theorem 6**  $L(\widehat{A}_{\text{SFW}}(A))^R = \text{SFW}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** Shortest forbidden words are strings  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in Q \setminus (Q_f \cup U)$  and are not synchronizing to  $Q \setminus Q_f$ , otherwise  $w$  would be a forbidden suffix. Therefore, using Lemma 5 and setting  $Q_1 = Q \setminus (Q_f \cup U)$  and  $Q_2 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_{nf}$ , we get that  $L(\widehat{A}(A, \widehat{q}_{nf}, \widehat{Q}_0))^R$  is the set of all the shortest forbidden words of  $A$ .  $\square$

**Theorem 7**  $L(\widehat{A}_{\text{SAW}}(A))^R = \text{SAW}(A)$  for a complete DFAA  $A = (Q, \Sigma, \delta, q_0, Q_f)$  with a set of useless states  $U \subseteq Q$ .

**Proof** Shortest allowed words are strings  $w \in \Sigma^*$  such that  $\delta^*(q_0, w) \in Q_f$  and are not synchronizing to  $Q_f \cup U$ , otherwise  $w$  would be an allowed suffix. Therefore, using Lemma 5 and setting  $Q_1 = Q_f$  and  $Q_2 = U$ , implying that  $\widehat{q}_0 = \widehat{q}_f$ , we get that  $L(\widehat{A}(A, \widehat{q}_f, \widehat{Q}_0))^R$  is the set of all the shortest allowed words of  $A$ .  $\square$

### 4.3 The size of the constructed DFAs

The shortest characteristic factors template automaton is a variant of a subset construction, similar to Rogers and Lambert [12], and its size is therefore exponential in the worst case. More precisely,  $|\widehat{Q}| \leq 3^{|Q|}$  for some  $\widehat{A}(A, \widehat{q}_0, \widehat{Q}_f) = (\widehat{Q}, \Sigma, \widehat{\delta}, \widehat{q}_0, \widehat{Q}_f)$  where  $A = (Q, \Sigma, \delta, q_0, Q_f)$ . Given the properties of its language, however, in each DFA for the shortest characteristic factors the part of the automaton containing reachable and useful states is proportional to the size of the resulting characteristic set, if the set is finite and, according to experiments of Rogers and Lambert [12], it is reasonable in practice.

[illegible]

## 5 Positive position run and its computation using pattern matching of characteristic factors

We define a positive run and a positive position run of a DFA, which represents all configurations and all positions in an input string, respectively, in which the automaton reaches a final state. An example can be found in Table 2.

**Definition 9** Let  $r = q_0, \dots, q_{|w|}$  be a run of a DFAA  $= (Q, \Sigma, \delta, q_0, Q_f)$  on a string  $w \in \Sigma^*$ . A *positive run* of the DFA on  $w$  is the set of all pairs  $(q_i, i)$  such that  $q_i \in Q_f$ . A *positive position run* of the DFA on  $w$  is the set of all indices  $i$  such that  $q_i \in Q_f$ .

We describe how the positive position run of a DFA can be computed with the use of exact pattern set matching of forbidden factors and of allowed suffixes in an input string, provided the automaton has its shortest characteristic factors finite, which is guaranteed for local automata. We can use any existing matching algorithm for the matching of a finite set of patterns, and the computation will share its properties. Because certain pattern-matching algorithms can skip parts of the input, the computation can be performed in sublinear time, which we demonstrate using the Commentz-Walter algorithm [16] with such a property. If the shortest characteristic sets are not finite, then the computation does not use pattern matching and the computation behaves as in the case of a standard implementation of the original automaton.

If a DFA has a finite set of the shortest characteristic factors, its language is strictly locally  $k$ -testable for some  $k \geq 1$ , and therefore according to Proposition 1 if it is to accept an input string of length at least  $k$ , none of its prefixes can its prefix cannot be forbidden, there can't be any occurrence of a forbidden factor and one of its suffixes has to be allowed or, equivalently, none of its suffixes can be forbidden. Finding an occurrence of a forbidden factor therefore means that the run has failed, and any occurrence of an allowed suffix before a forbidden factor is found indicates that the automaton reached a final state. Similarly, occurrence of a forbidden suffix without any prior forbidden factor indicates an automaton has reached a useful non-final state, therefore not finding an occurrence of a forbidden suffix before a forbidden factor indicates the automaton had to reach a final state. If the characteristic factors are finite, it is possible to use some matching algorithm for a finite set of patterns for such task. The prefix is sufficient to check using the standard DFA run. The choice of whether to use forbidden or allowed suffixes can be made, for example, based on which shortest characteristic set is smaller. This is covered in Algorithm 2. Figure 5 shows example of preprocessed trie of patterns for DFAA in Fig. 3 extracted from automata  $\hat{A}_{\text{SFF}}(A)$  and  $\hat{A}_{\text{SAS}}(A)$  in Fig. 4 and Table 2 shows the matching phase of Algorithm 2 with the use of the Commentz-Walter algorithm.

**Theorem 8** *Given a complete DFAA and an input string  $w$  Algorithm 2 correctly computes the positive position run of  $A$  on  $w$ .*

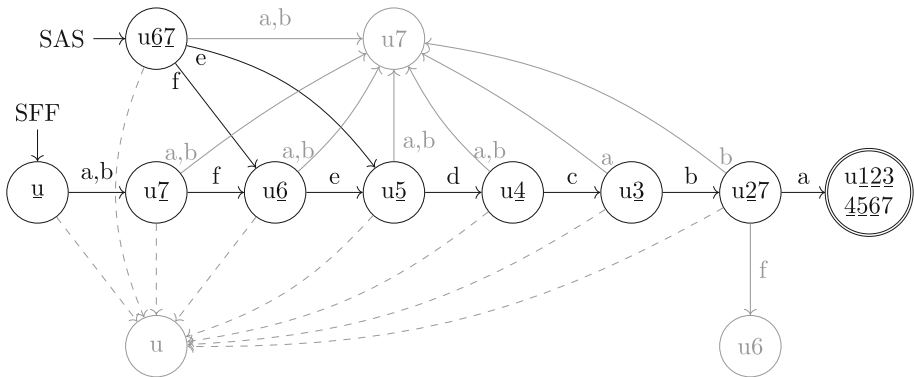
**Proof** First, Algorithm 1 is used to compute automata for the shortest characteristic factors. Based on the bound of sizes of characteristic sets it also determines some upper bound  $k$  for which  $L(A)$  is strictly locally  $(k + 1)$ -testable and this bound is minimal if  $A$  is minimal DFA. Such bound is finite only if automata for the shortest characteristic factors do not contain a cycle on reachable and useful states, and equals either one less than the maximum length of all the shortest forbidden factors or the maximum length of all the shortest forbidden suffixes, prefixes, and words.

Next a standard run of the DFA is performed (starting on line 2) on the prefix of length up to  $k$  while marking positions where a final state was reached. If a useless state is encountered,

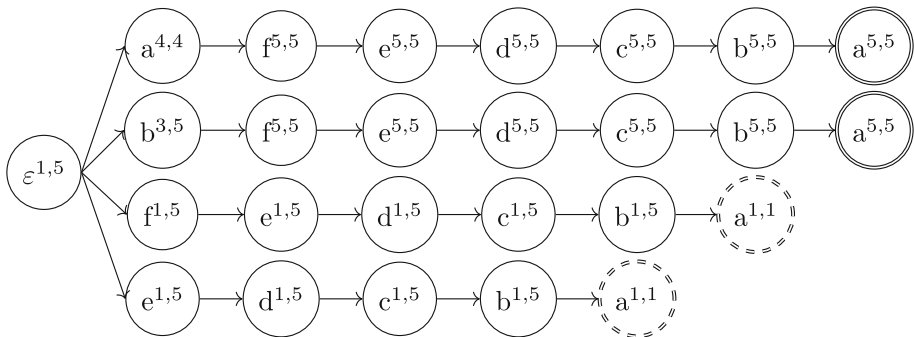


**Algorithm 2:** Computing positive position run of DFA

**Require:** Complete DFAA =  $(Q, \Sigma, \delta, q_0, Q_f)$  with useless states  $U \subseteq Q$ , input string  $w$   
**Ensure :** Positive position run PPR of  $A$  on  $s$ , i.e. set of indices where  $A$  reaches a final state in  $w$   
1 Using Algorithm 1 with  $\hat{Q}_i = \{\hat{q}_U, \hat{q}_f, \hat{q}_{nf}\}$  obtain  $\hat{A}_{SFF}$ ,  $\hat{A}_{SFS}$ ,  $\hat{A}_{SAS}$ ,  $\hat{A}_{SFW}$ ,  $\hat{A}_{SAW}$  of  $A$ .  
2 **if** either DFAA contains cycle **then**  $k \leftarrow \infty$  **else**  
 $k \leftarrow \max(\{|p| - 1 \mid p \in L(\hat{A}_{SFF})\} \cup \{|p| \mid p \in L(\hat{A}_{SFP}) \cup L(\hat{A}_{SFS}) \cup L(\hat{A}_{SFW})\})$   $q \leftarrow q_0$   
3 **if**  $q_0 \in Q_f$  **then** PPR  $\leftarrow \{0\}$  **if**  $q_0 \in U$  **then return** **for**  $i \leftarrow 1$  **to**  $\min(|w|, k)$  **do**  
4      $q \leftarrow \delta(q, w[i])$   
5     **if**  $q \in Q_f$  **then** PPR  $\leftarrow \text{PPR} \cup \{i\}$  **if**  $q \in U$  **then return**  
6 **if**  $|w| \leq k$  **then return** // DFA for non-SLTL never gets past here Using a pattern set matching algorithm match SFF( $A$ ) and either SAS( $A$ ) or SFS( $A$ ) in  $w$ . If a forbidden factor is matched, then end. When matching allowed suffixes, if a match is found then add the end position index to PPR. When matching forbidden suffixes, if no match ending at some position is found then add the index to PPR.



**Fig. 4** DFAs  $\hat{A}_{SFF}(A)$  and  $\hat{A}_{SAS}(A)$  of DFAA in Fig. 3. Dashed transitions are for all symbols without explicit transition. Gray states are useless states and their further transitions are omitted. Bottom dash under some  $q$  within a state  $\hat{q}$  indicates  $\hat{q}(q) = 1$



**Fig. 5** Trie of the reverse of the shortest forbidden factors and the shortest allowed suffixes of the DFA in Fig. 3 extracted from DFAs in Fig. 4. Solid final states represent forbidden patterns while dashed ones represent allowed patterns. Numbers in each node represent values of functions shift1 and shift2 (see Appendix A)

then a final state can no longer be reached and the computation terminates. In case the string is not longer than  $k$  or  $A$  is not local, making  $k$  unbounded, the positive position run is correctly computed with this step. For any string longer than  $k$  this step checks conditions 1 and 2 of Proposition 1.

For positions beyond  $k$  the automaton reaches a final state on a position  $i$  if a prefix of  $w$  of length  $i$  is in  $L(A)$ . For that it further needs to hold that none of its factors are forbidden to satisfy condition 2, and it must either contain an allowed suffix or not contain any forbidden suffixes to satisfy condition 4. A pattern set matching algorithm is therefore used to find occurrences of the shortest forbidden factors, which terminates the computation, and either the shortest allowed or forbidden suffixes. To reach a final state on  $i$  no forbidden factor can be matched up to this point and either an allowed suffix ending at  $i$  must be matched or a forbidden suffix ending at  $i$  cannot be matched. Marking such occurrences then completes the positive position run.  $\square$

## 6 Conclusion and future work

We have presented a simplified method of extraction of the shortest characteristic factors of a DFA and also generalized it for all input DFAs. In our constructions we have not considered the shortest allowed factors and prefixes as we are not certain how such factors could be characterized in respect to the input DFA.

We have also presented a new algorithm for computing the positive position run of a DFA, which uses exact finite set pattern matching provided the DFA has those characteristic factors finite, which is guaranteed for local DFAs. The positive run of a DFA, which also provides the final states in the run of the DFA, can be computed in a similar way: Instead of constructing the single automaton  $\hat{A}_{SAS}$  with its initial state containing all the final states of the original automaton, we would construct automata for each final state separately, which means having for each of them a separate initial state in the shortest characteristic factors template automaton. For such construction it is sufficient to supply all of these different initial states to Algorithm 1.

Synchronization and locality have also recently been generalized for tree automata [21, 22]. The generalization of concepts presented in this paper for tree automata can be a topic of future work.

## Appendix A: Commentz-Walter algorithm

This section describes the Commentz-Walter string pattern set matching algorithm [16] which is used for demonstration purposes in the paper.

A *trie* over alphabet  $\Sigma$  is a rooted labeled tree  $T = (V, E)$  such that:

- For a node  $v \in V$  label  $l(v) \in \Sigma$  or  $l(v) = \varepsilon$  if  $v$  is root.
- For each children  $v_1, v_2 \in \text{children}(v)$  of a node  $v \in V$ :  $l(v_1) \neq l(v_2)$ .

For a node  $v \in V$  and a set of patterns  $K$  let:

- $w_{\min} = \min(\{|w| \mid w \in K\})$
- $\text{depth } d(v) = \begin{cases} 0 & \text{if } v \text{ is root} \\ d(v') + 1 & \text{if } v' \text{ is son of } v \end{cases}$
- string  $w(v) = l(v_0)l(v_1) \dots l(v_m)$ , where  $v_0$  is root,  $v_m = v$  and  $v_{i+1}$  is son of  $v_i$ ,  $i < m$

- $out(v) = \{w(v) \in K\}$
- $set1(v) = \{v' \mid w(v') = uw(v)\}$  for some non empty string  $u$
- $set2(v) = \{v' \mid v' \in set1(v) \wedge w(v') \in K\}$
- $shift1(v) = \begin{cases} 1 & \text{if } v \text{ is root} \\ \min(\{d(v') - d(v) \mid v' \in set1(v)\} \cup \{w_{min}\}) & \text{else} \end{cases}$
- $shift2(v) = \begin{cases} w_{min} & \text{if } v \text{ is root} \\ \min(\{d(v') - d(v) \mid v' \in set2(v)\} \cup \{shift2(v_p)\}) & \text{else} \end{cases}$ , where  $v_p$  is parent of  $v$
- for  $a \in \Sigma : char(a) = \min(\{d(v') \mid l(v') = a\} \cup \{w_{min} + 1\})$

For our use case we also differentiate two types of patterns: forbidden and allowed. So for the purposes of this paper we also add boolean functions  $allowed(v)$  and  $forbidden(v)$  which are true if  $w(v)$  is a pattern of respective type. We further modify the matching algorithm to stop if a forbidden pattern is matched. Algorithm 3 describes the Commentz-Walter algorithm with said modifications.

---

### Algorithm 3: Commentz-Walter algorithm

---

**Require:** Input string  $t$  of size  $n$ , trie of reversed patterns  $T$   
**Ensure :** set of indices FI where an allowed pattern is matched before an occurrence of forbidden one

```

1  $v \leftarrow \text{root of } T; i \leftarrow w_{min} - 1; j \leftarrow 0$ 
2 while  $i < n$  do
3   while  $\exists v' \in children(v) : l(v') = t[i]$  do
4      $v \leftarrow v'$ 
5      $j \leftarrow j + 1$ 
6     if  $forbidden(v)$  then return if  $allowed(v)$  then  $FI \leftarrow FI \cup \{i\}$ 
7      $i \leftarrow i + \min(\max(shift1(v), char(t[i - j]) - j - 1), shift2(v))$ 
8      $j \leftarrow 0$ 
```

---

**Acknowledgements** The authors acknowledge the support of the OP VVV MEYSfunded project CZ.02.1.01/0.0/0.0/16\_019/0000765 “Research Center for Informatics”, of the Grant Agency of the Czech Technical University in Prague, Grant Number SGS20/208/OHK3/3T/18, of the Swedish Research Council under grant no. 2020-03852, and of the Wallenberg AI, Autonomous Systems and Software Program through the NEST project STING—Synthesis and analysis with Transducers and Invertible Neural Generators.

**Author contributions** Š.P. is the main author of the idea of creating the automata accepting the shortest characteristic factors. J.J. is the main author of the idea of the positive position run of the automaton and of the use of pattern matching for the computation of that run. Š.P. and J.J. wrote together main definitions and the Introduction. Š.P. wrote the other main manuscript content. Both authors reviewed the manuscript.

**Funding** Open access funding provided by Umea University.

**Data availability** No datasets were generated or analysed during the current study.

## Declarations

**Conflict of interest** The authors declare no competing interests.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the

article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

1. Crochemore, M., Hancart, C.: Pattern matching in strings. In: Algorithms and Theory of Computation Handbook. Chapman & Hall/CRC Applied Algorithms and Data Structures series. CRC Press, Boca Raton (1999). <https://doi.org/10.1201/9781420049503-c12>
2. Sipser, M.: Introduction to the Theory of Computation, 3rd edn. Course Technology, Melbourne (2013)
3. Baeza-Yates, R.A.: A unified view to string matching algorithms. In: SOFSEM 1996. Lecture Notes in Computer Science, vol. 1175, pp. 1–15. Springer, Cham (1996). <https://doi.org/10.1007/BFb0037393>
4. Melichar, B.: String matching with  $k$  differences by finite automata. In: ICPR 1996, Vienna, Austria, pp. 256–260. IEEE Computer Society, Piscataway (1996). <https://doi.org/10.1109/ICPR.1996.546828>
5. Melichar, B., Holub, J.: 6d classification of pattern matching problems. In: Proceedings of the Prague Stringology Club Workshop 1997, pp. 24–32 (1997)
6. McNaughton, R., Papert, S.: Counter-Free Automata, vol. 65, p. 163. MIT Press, Cambridge (1971)
7. Béal, M.-P., Senellart, J.: On the bound of the synchronization delay of a local automaton. Theor. Comput. Sci. **205**(1), 297–306 (1998). [https://doi.org/10.1016/S0304-3975\(98\)80011-X](https://doi.org/10.1016/S0304-3975(98)80011-X)
8. Caron, P.: Families of locally testable languages. Theor. Comput. Sci. **242**(1), 361–376 (2000). [https://doi.org/10.1016/S0304-3975\(98\)00332-6](https://doi.org/10.1016/S0304-3975(98)00332-6)
9. Zalcstein, Y.: Locally testable languages. J. Comput. Syst. Sci. **6**(2), 151–167 (1972). [https://doi.org/10.1016/S0022-0000\(72\)80020-5](https://doi.org/10.1016/S0022-0000(72)80020-5)
10. Béal, M., Crochemore, M., Mignosi, F., Restivo, A., Sciortino, M.: Computing forbidden words of regular languages. Fundam. Informatica **56**(1–2), 121–135 (2003)
11. Rogers, J., Lambert, D.: Extracting forbidden factors from regular stringsets. In: Proceedings of the 15th Meeting on the Mathematics of Language, pp. 36–46. Association for Computational Linguistics, London (2017). <https://doi.org/10.18653/v1/W17-3404>
12. Rogers, J., Lambert, D.: Extracting subregular constraints from regular stringsets. J. Lang. Model. **7**(2), 143 (2019). <https://doi.org/10.15398/jlm.v7i2.209>
13. Apostolico, A., Galil, Z. (eds.): Pattern Matching Algorithms. Oxford University Press, New York (1997). <https://global.oup.com/academic/product/pattern-matching-algorithms-9780195113679>
14. Crochemore, M., Rytter, W.: Text Algorithms. Oxford University Press, New York (1994). <http://www-igm.univ-mlv.fr/%TEmac/REC/B1.html>
15. Cleophas, L.G., Watson, B.W., Zwaan, G.: A new taxonomy of sublinear right-to-left scanning keyword pattern matching algorithms. Sci. Comput. Program. **75**(11), 1095–1112 (2010). <https://doi.org/10.1016/j.scico.2010.04.012>
16. Commentz-Walter, B.: A string matching algorithm fast on the average. In: Automata, Languages and Programming, pp. 118–132. Springer, Berlin, Heidelberg (1979). [https://doi.org/10.1007/3-540-09510-1\\_10](https://doi.org/10.1007/3-540-09510-1_10)
17. Watson, B.W., Zwaan, G.: A taxonomy of sublinear multiple keyword pattern matching algorithms. Sci. Comput. Program. **27**(2), 85–118 (1996). [https://doi.org/10.1016/0167-6423\(96\)00008-1](https://doi.org/10.1016/0167-6423(96)00008-1)
18. Watson, B.W.: Taxonomies and toolkits of regular language algorithms. PhD thesis, Mathematics and Computer Science, Eindhoven (1995). <https://doi.org/10.6100/IR444299>
19. Crochemore, M., Hancart, C.: Automata for matching patterns. In: Handbook of Formal Languages, Volume 2. Linear Modeling: Background and Application, pp. 399–462. Springer, Berlin Heidelberg (1997). [https://doi.org/10.1007/978-3-662-07675-0\\_9](https://doi.org/10.1007/978-3-662-07675-0_9)
20. Janoušek, J., Plachý, Š.: Shortest characteristic factors of a deterministic finite automaton and computing its positive position run by pattern set matching. In: Fernau, H., Gaspers, S., Klasing, R. (eds.) SOFSEM 2024: Theory and Practice of Computer Science, pp. 326–339. Springer, Cham (2024)
21. Blažej, V., Janoušek, J., Plachý, Š.: On the smallest synchronizing terms of finite tree automata. In: CIAA 2023, pp. 79–90. Springer, Cham (2023). [https://doi.org/10.1007/978-3-031-40247-0\\_5](https://doi.org/10.1007/978-3-031-40247-0_5)
22. Plachý, Š., Janoušek, J.: On synchronizing tree automata and their work—optimal parallel run, usable for parallel tree pattern matching. In: SOFSEM 2020, pp. 576–586. Springer, Cham (2020). [https://doi.org/10.1007/978-3-030-38919-2\\_47](https://doi.org/10.1007/978-3-030-38919-2_47)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.